

Arquitetura

Este documento descreve a arquitetura do Sistema de Roletas Personalizadas, detalhando suas camadas, componentes principais, tecnologias utilizadas e o fluxo de dados. O objetivo é fornecer uma visão clara de como o sistema está estruturado e como suas partes interagem para atender aos requisitos definidos.

4.1. Visão Geral da Arquitetura

O Sistema de Roletas Personalizadas segue uma arquitetura em camadas (Apresentação, API, Domínio e Persistência) baseada em uma **API REST** implementada com Django REST Framework (DRF). O frontend, escrito em HTML5, CSS3 e JavaScript puro (Vanilla JS), é servido pela própria aplicação Django e consome a API por meio de requisições JSON. Os dados são armazenados em um banco PostgreSQL gerenciado pelo Supabase e a aplicação é publicada em um ambiente de deploy público.

4.2. Diagrama de Componentes e Implantação (UML)

O diagrama abaixo ilustra os principais componentes do sistema e sua implantação: o navegador do usuário, o servidor de aplicação Django (com as camadas de apresentação, API, domínio e persistência) e o servidor de banco de dados PostgreSQL no Supabase.

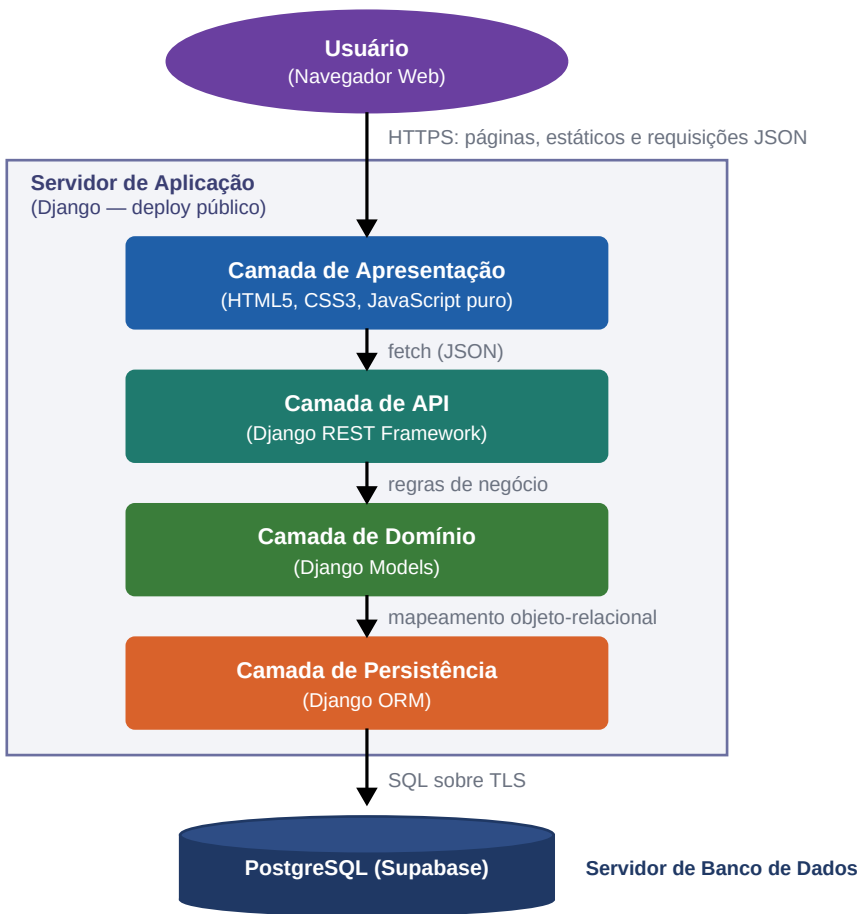


Figura 1 — Componentes e implantação do Sistema de Roletas Personalizadas.

4.3. Descrição das Camadas e Componentes

A arquitetura é dividida nas seguintes camadas de responsabilidade:

1. Camada de Apresentação (Frontend):

- **Responsabilidade:** apresentar a interface ao usuário (UI) e coletar suas interações. Exibe as telas de login, cadastro, lista de roletas (dashboard) e a própria roleta, em layout responsivo e com modo claro/escuro.
- **Tecnologias:** HTML5, CSS3 e JavaScript puro (Vanilla JS).
- **Integração:** o JavaScript consome a API REST via fetch (JSON), atualiza o DOM de forma dinâmica e controla as animações da roleta (giro, sons e confetes) sem depender de bibliotecas externas complexas.

2. Camada de API e Aplicação (Backend — Lógica):

- **Responsabilidade:** receber as requisições HTTP da camada de apresentação, autenticar o usuário, validar dados, aplicar as regras de negócio e garantir que cada usuário acesse apenas os seus próprios recursos.
- **Tecnologias:** Python 3.13+, Django e Django REST Framework (views/viewsets e serializers). A autenticação é baseada em sessão do Django, com proteção CSRF.
- **Integração:** os serializers validam as regras descritas no diagrama de classes (mínimo de 2 itens, unicidade de nomes, pesos positivos) e as views delegam o sorteio à camada de domínio.

Endpoints sugeridos:

Recurso	Operações	Caso de uso
/api/usuarios/	POST (cadastro)	UC01
/api/auth/login/ e /api/auth/logout/	POST	UC02
/api/roletas/	GET (lista), POST (cria roleta e itens)	UC03, UC04, UC05, UC06
/api/roletas/{id}/	GET, PUT/PATCH (edita), DELETE (exclui)	UC04, UC05, UC06, UC07
/api/roletas/{id}/girar/	POST (sorteio)	UC07
/api/usuarios/me/	PATCH (preferência dark_mode)	UC08

3. Camada de Domínio (Backend — Regras de Negócio):

- **Responsabilidade:** representar as entidades Usuario, Roleta e ItemRoleta e executar as regras do domínio, incluindo o sorteio ponderado pelos pesos (peso nulo equivale a 1).
- **Tecnologias:** Django Models.
- **Integração:** o método girar() da Roleta é executado no backend, desconsidera os itens já sorteados na sessão de giro quando remover_fatia_sorteada está ativa e retorna o item vencedor.

4. Camada de Persistência (Backend — Dados):

- **Responsabilidade:** traduzir os objetos de domínio (Python) para operações em banco de dados e garantir a integridade relacional.
- **Tecnologias:** Django ORM (Object-Relational Mapping).
- **Integração:** o ORM mapeia as classes Usuario, Roleta e ItemRoleta para as tabelas usuario, roleta e item_roleta, aplicando as chaves estrangeiras (usuario_id, roleta_id) e as restrições de integridade (como ON DELETE CASCADE e as restrições de unicidade). Gravações compostas usam transações atômicas.

5. Servidor de Banco de Dados:

- **Responsabilidade:** armazenamento seguro, persistente e isolado dos dados do sistema.
- **Tecnologias:** PostgreSQL gerenciado pelo Supabase, acessado pela aplicação por conexão segura (TLS), com credenciais mantidas em variáveis de ambiente.

4.4. Fluxos de Dados Principais

Os fluxos a seguir detalham as interações entre os componentes do sistema durante as duas operações mais críticas:

Fluxo 1: Criação de Nova Roleta e Itens (UC04 e UC06)

Este fluxo descreve o caminho dos dados desde a entrada do usuário até o armazenamento persistente (a edição de uma roleta existente segue o mesmo caminho, com PUT/PATCH):

1. **Ação do Usuário:** no navegador, o usuário logado acessa a tela “Criar Roleta”, informa o nome, personaliza cores e configurações e adiciona as fatias (itens). Ao clicar em “Salvar”, o JavaScript envia os dados em JSON via requisição POST à API.
2. **Recepção no Servidor:** a requisição HTTPS chega à aplicação Django e a view/viewset do DRF correspondente é acionada.
3. **Validação:** o DRF valida a sessão do usuário (garantindo que ele está autenticado) e o serializer valida os dados (nome informado e único para o usuário, no mínimo 2 itens, textos de itens únicos, pesos maiores que zero).
4. **Processamento no Domínio:** os objetos Roleta e múltiplos objetos ItemRoleta são instanciados em memória pelo Django, vinculados ao usuário logado.
5. **Persistência (ORM):** o Django ORM gera as instruções SQL INSERT necessárias dentro de uma transação atômica, para evitar roletas incompletas.
6. **Gravação no Banco:** o PostgreSQL executa os INSERTs, criando os registros nas tabelas roleta e item_roleta e mantendo as chaves estrangeiras com o usuário logado e entre a roleta e seus itens.
7. **Resposta:** a API retorna sucesso (201 Created, com a roleta em JSON). O JavaScript redireciona o usuário para a lista de roletas (Dashboard), exibindo uma mensagem de sucesso. Em caso de erro de validação (400), os campos inválidos são destacados e nada é gravado.

Fluxo 2: Acionamento e Sorteio da Roleta (UC07)

Este fluxo descreve como a lógica de sorteio (backend) e a animação (frontend) interagem:

1. **Ação do Usuário:** na tela da roleta, o usuário clica no botão de giro (“Girar” ou o texto definido em texto_botao_giro).
2. **Requisição de Sorteio (Backend):** o botão aciona uma requisição POST (fetch) ao endpoint de sorteio da roleta. Quando remover_fatia_sorteada está ativa, o JavaScript envia também os itens já sorteados na sessão de giro. O servidor processa os pesos dos itens ativos, sorteia um resultado e o devolve ao frontend via JSON. A lógica de sorteio fica sempre no backend, preservando a integridade do resultado.
3. **Animção (Vanilla JS):** com o item vencedor já determinado, o script em JavaScript puro calcula o ângulo final necessário e inicia a animação de rotação (manipulando propriedades CSS como transform: rotate(...) ou desenhando em um <canvas>).
4. **Efeitos:** durante a animação, o JavaScript dispara a reprodução dos arquivos de áudio (se som_giro_enabled for verdadeiro).
5. **Conclusão e Resultado:** quando a animação atinge o ângulo da fatia vencedora, a roleta para. O JavaScript aciona os efeitos de comemoração (confetes via DOM/Canvas, se efeito_vitoria_enabled for verdadeiro) e exibe o modal com o texto do item sorteado em destaque.

6. **Pós-sorteio:** se `remover_fatia_sorteada` for verdadeiro, o item sorteado é retirado apenas da lista ativa da sessão de giro, sem alteração no banco de dados. Quando restarem menos de 2 itens ativos, o botão de giro é desabilitado e o usuário pode reiniciar a roleta.

4.5. Limites e Premissas Tecnológicas

- **Implantação:** a aplicação Django (frontend e API) é publicada em um ambiente de deploy público, enquanto o banco de dados PostgreSQL é hospedado no Supabase. Para desenvolvimento, a aplicação pode ser executada em localhost. Nesta versão acadêmica não há configurações de balanceamento de carga nem de orquestração de contêineres.
- **Integrações:** o Supabase (PostgreSQL gerenciado) é o único serviço externo utilizado; o sistema não possui dependências de APIs de terceiros. A autenticação e a gestão de dados são implementadas pela própria aplicação.
- **Segurança:** comunicação via HTTPS, senhas armazenadas com hash (PBKDF2/Argon2), proteção CSRF e isolamento dos dados por usuário; credenciais e chaves ficam em variáveis de ambiente.